# Calculating Correct and Efficient Programs in Rocq

David Castro-Perez

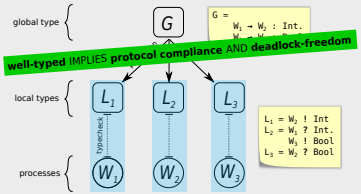
David.CastroPerez@rhul.ac.uk




Based on joint work with Marco Paviotti and Michael Vollmer (University of Kent):
*Program Optimisations via Hylomorphisms for Extraction of Executable Code*, ITP 2025.

# My Research in One Slide

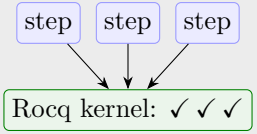
## Protocols as types




well-typed ⇒ no deadlocks

POPL'19, OOPSLA'20, ECOOP'23, POPL'26

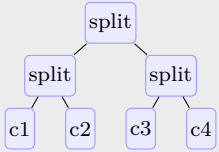
## Certified tools




mechanised metatheory, certified languages

TACAS'20, PLDI'21, ITP'25

## Parallelism

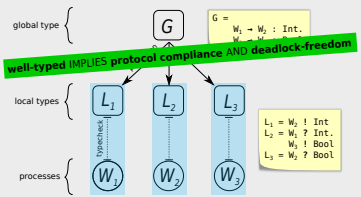



structured parallelism, *predicted* speedups

ICFP'16, FGCS'18, CC'20

# My Research in One Slide

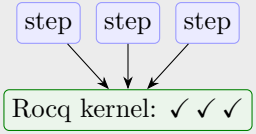
### Protocols as types




well-typed ⇒ no deadlocks

POPL'19, OOPSLA'20, ECOOP'23, POPL'26

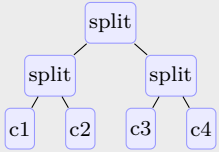
### Certified tools




mechanised metatheory, certified languages

TACAS'20, PLDI'21, ITP'25

### Parallelism

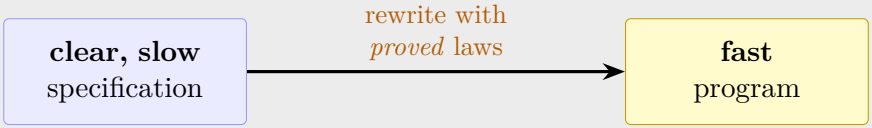



structured parallelism, *predicted* speedups

ICFP'16, FGCS'18, CC'20

**Today**: certified tools + hylomorphisms → fast, correct code

## In One Slide




**The idea** (1980s):

- Programs as algebra
- Optimise by rewriting
- Correct by construction

**Our work:**

- Every step machine-checked
- No extra axioms
- Runnable code out

# The Problem

Fast *and* Correct

# A Specification and a Program



# A Specification and a Program




```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

# A Specification and a Program




```
let rec sort_times_two = function
  | [] -> []
  | h :: t ->
    let (l, r) = partition (fun x -> x < h) t in
    sort_times_two l @ (h * 2) :: sort_times_two r
```

Are they equal? Can we *derive* the second from the first?

## Three Ways to Fast and Correct Code

| Approach | Good | But |
|---|---|---|
| Write it, then **verify** it | any program | a new proof for every program |

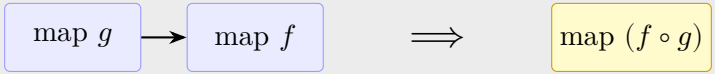
## Three Ways to Fast and Correct Code

| **Approach** | **Good** | **But** |
|---|---|---|
| Write it, then **verify** it | any program | a new proof for every program |
| **Verified compiler** | translation is correct | keeps your algorithm: $O(n^3)$ stays $O(n^3)$ |

## Three Ways to Fast and Correct Code

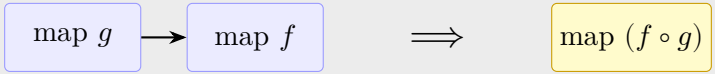
| **Approach** | **Good** | **But** |
|---|---|---|
| Write it, then **verify** it | any program | a new proof for every program |
| **Verified compiler** | translation is correct | keeps your algorithm: $O(n^3)$ stays $O(n^3)$ |
| **Calculate** it from a spec | proof comes with the derivation | needs laws that *apply* |

# You Already Trust Rewrite Rules



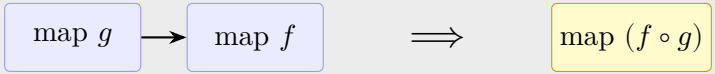

- ML compilers *fuse* tensor operations into one kernel.

# You Already Trust Rewrite Rules



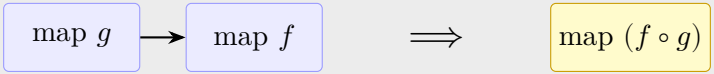

- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.

# You Already Trust Rewrite Rules




- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.
- Compilers inline, fuse loops, remove temporary data.

# You Already Trust Rewrite Rules

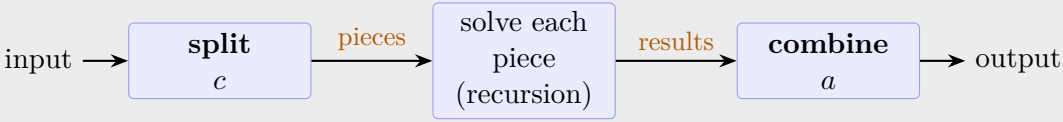



- ML compilers *fuse* tensor operations into one kernel.
- Database engines rewrite query plans.
- Compilers inline, fuse loops, remove temporary data.
- Usually: a **fixed** set of rules, **trusted**, not proved.

# The Laws

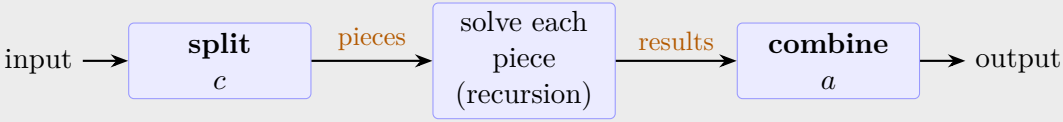
Divide and Conquer as Algebra

# One Shape for Many Algorithms



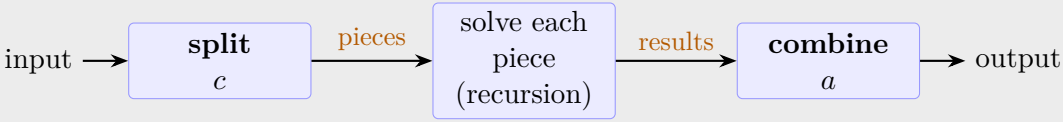

| **Algorithm** | **split** $c$ | **combine** $a$ |
|---|---|---|

# One Shape for Many Algorithms



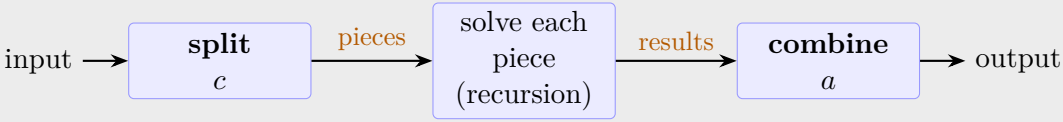

| **Algorithm** | **split** $c$ | **combine** $a$ |
|---|---|---|
| quicksort | partition around a pivot | concatenate |

# One Shape for Many Algorithms



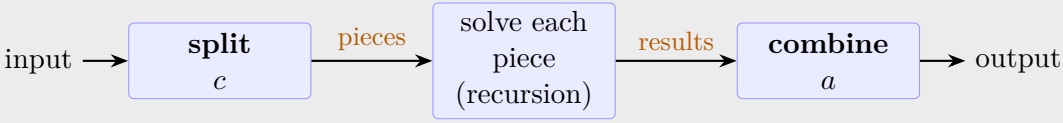

| **Algorithm** | **split** $c$ | **combine** $a$ |
|---|---|---|
| quicksort | partition around a pivot | concatenate |
| mergesort | cut in half | merge |

# One Shape for Many Algorithms




| **Algorithm** | **split** $c$ | **combine** $a$ |
|---|---|---|
| quicksort | partition around a pivot | concatenate |
| mergesort | cut in half | merge |
| $x^n$ | $n \mapsto$ last bit, $n/2$ | square, maybe multiply by $x$ |

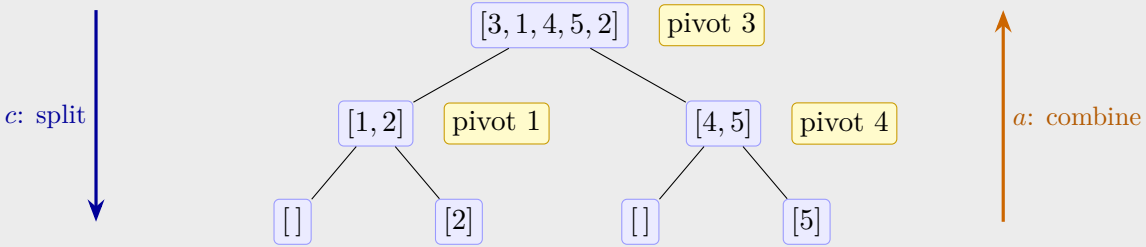
# One Shape for Many Algorithms



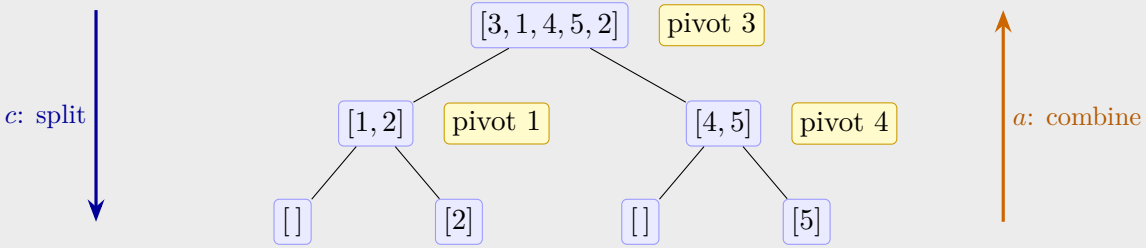

| **Algorithm** | **split** $c$ | **combine** $a$ |
|---|---|---|
| quicksort | partition around a pivot | concatenate |
| mergesort | cut in half | merge |
| $x^n$ | $n \mapsto$ last bit, $n/2$ | square, maybe multiply by $x$ |

**Hylomorphism** $[\![a, c]\!]$: the recursion is *generic*; only $a$ and $c$ change.

# The Call Tree of Quicksort

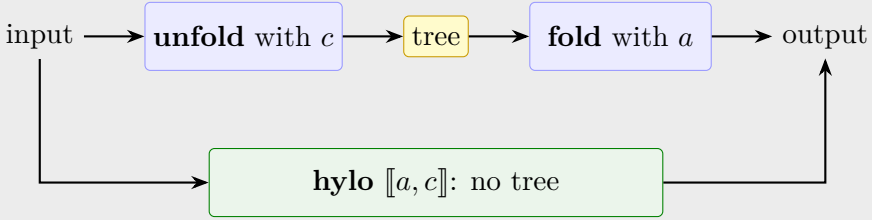


# The Call Tree of Quicksort



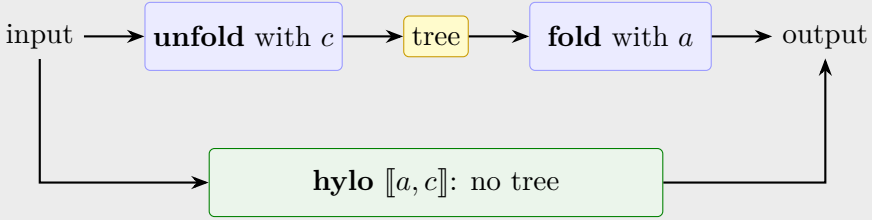

The tree is *virtual*: it is never built in memory.

# Producers and Consumers




- Specification: *build*, then *consume*. Easy to read.

# Producers and Consumers




- Specification: *build*, then *consume*. Easy to read.
- Program: do both at once. Fast.

## Two Laws

**Fusion** $[\![a, c]\!] \rightarrow h = [\![b, c]\!]$

if $h$ distributes over $a$

**Deforestation** $[\![\mathsf{build}, c]\!] \rightarrow [\![a, \mathsf{take_apart}]\!] = [\![a, c]\!]$

- Proved **once**, for every split, combine, data type.

## Two Laws

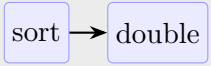
**Fusion** $[\![a, c]\!] \rightarrow h = [\![b, c]\!]$

if $h$ distributes over $a$

**Deforestation** $[\![\mathsf{build}, c]\!] \rightarrow [\![a, \mathsf{take_apart}]\!] = [\![a, c]\!]$

- Proved **once**, for every split, combine, data type.
- Each use is a *rewrite step*.

# Deriving the Example



# Deriving the Example

$$\mathrm{sort} \rightarrow \mathrm{double}$$

$$= \quad [\![\mathsf{concat}, \mathsf{partition}]\!] \rightarrow \mathrm{double} \qquad \text{definition of quicksort}$$

# Deriving the Example

$$\mathsf{sort} \rightarrow \mathsf{double}$$

$$= \quad [\![\mathsf{concat}, \mathsf{partition}]\!] \rightarrow \mathsf{double} \qquad \text{definition of quicksort}$$

$$= \quad [\![\mathsf{concat}\ \text{and double pivot}, \mathsf{partition}]\!] \qquad \text{fusion}$$

# Deriving the Example

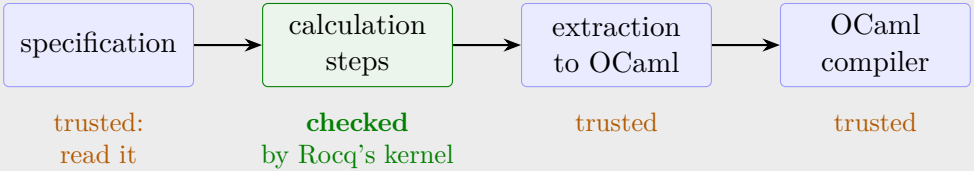
$$\begin{aligned}
&\boxed{\text{sort}} \rightarrow \boxed{\text{double}} \\
=\ &\boxed{[\![\mathsf{concat}, \mathsf{partition}]\!]} \rightarrow \boxed{\text{double}} && \text{definition of quicksort} \\
=\ &\boxed{[\![\mathsf{concat}\text{ and double pivot}, \mathsf{partition}]\!]} && \text{fusion}
\end{aligned}$$

Each $=$ is a **theorem** applied by a proof assistant.

# Machine-Checked

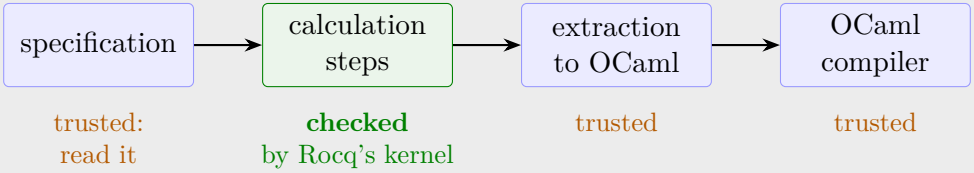
Hylomorphisms in Rocq

# What You Have to Trust




- Who picked the steps does not matter: a person, a script, a model.

# What You Have to Trust




- Who picked the steps does not matter: a person, a script, a model.
- A wrong step is **rejected**, not miscompiled.

# Three Obstacles

**Termination**

every function must provably stop

**No extra axioms**

comparing functions needs one

**Clean code**

generic proofs extract to unsafe casts

## Three Obstacles

**Termination**

every function must provably stop

**No extra axioms**

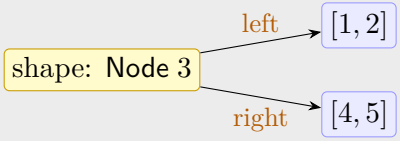
comparing functions needs one

**Clean code**

generic proofs extract to unsafe casts

Haskell has the laws but not termination; Rocq has termination but rejects the textbook definitions.

# Our Fix: Shapes and Positions

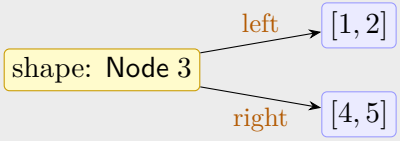



shape: Leaf no positions

| valid? | left | right |
|---|---|---|
| `Node h` | yes | yes |
| `Leaf` | no | no |

- One type of positions + a **yes/no check**: plain OCaml types.

# Our Fix: Shapes and Positions




shape: Leaf no positions

| valid? | left | right |
|---|---|---|
| `Node h` | yes | yes |
| `Leaf` | no | no |

- One type of positions + a **yes/no check**: plain OCaml types.
- Proofs about the check are **erased** from the code.

# Termination: Recursion on a Proof




- Proved **once per split**, e.g. “pieces are shorter”.

# Termination: Recursion on a Proof




- Proved **once per split**, e.g. “pieces are shorter”.
- The proof is scaffolding: **erased** at extraction.

# No Extra Axioms




i.e. no axioms at all

- Equality = *same results on equal inputs* (setoids).

# No Extra Axioms

```
Print Assumptions mss_calc.
```

→ **Closed under the global context**

i.e. no axioms at all

- Equality = *same results on equal inputs* (setoids).
- Laws become *rewrite* steps.

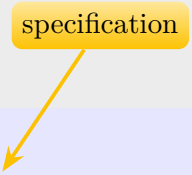
# In Practice

From Calculation to Code

# A Calculation in Rocq

```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  finished.
Qed.
```

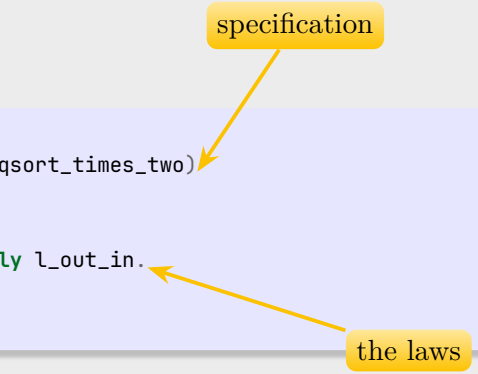
# A Calculation in Rocq




```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  finished.
Qed.
```

# A Calculation in Rocq

```
Derive qsort_times_two
  in (Lmap times_two \o cata merge \o rana tsplit =e qsort_times_two)
  as qsort_times_two_calc.
Proof.
  rewrite map_merge_fusion, hylo_ana, deforest by apply l_out_in.
  finished.
Qed.
```



# Quicksort, Step by Step

```
(Lmap times_two \o cata merge) \o rana tsplit
```

# Quicksort, Step by Step

```
  (Lmap times_two \o cata merge) \o rana tsplit
=e  (* map_merge_fusion: double inside merge *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o rana tsplit
```

# Quicksort, Step by Step

```
  (Lmap times_two \o cata merge) \o rana tsplit
=e  (* map_merge_fusion: double inside merge *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o rana tsplit
=e  (* hylo_ana: the unfold is a hylo *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o hylo l_in tsplit
```

# Quicksort, Step by Step

```
  (Lmap times_two \o cata merge) \o rana tsplit
=e  (* map_merge_fusion: double inside merge *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o rana tsplit
=e  (* hylo_ana: the unfold is a hylo *)
  hylo (merge \o natural (nt_shape id times_two)) l_out \o hylo l_in tsplit
=e  (* deforest: build, then take apart, cancel out *)
  hylo (merge \o natural (nt_shape id times_two)) tsplit
```

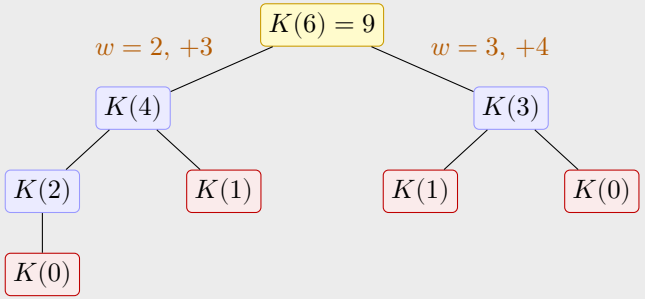
# Quicksort: What Comes Out

```
let rec qsort_times_two_code = function
| [] -> []
| h :: t ->
  let (l, r) = partition (fun x0 -> Uint63.lt x0 h) t in
  let x0 = fun p ->
    qsort_times_two_code (match p with
                          | Lbranch -> l
                          | Rbranch -> r)
  in
  app (x0 Lbranch) ((Uint63.mul (Uint63.of_int (2)) h) :: (x0 Rbranch))
```

The last line, extracted: **one pass**, doubling each pivot as it goes. The hand-written program from the start, now with a proof.

# Dynamic Programming: Knapsack

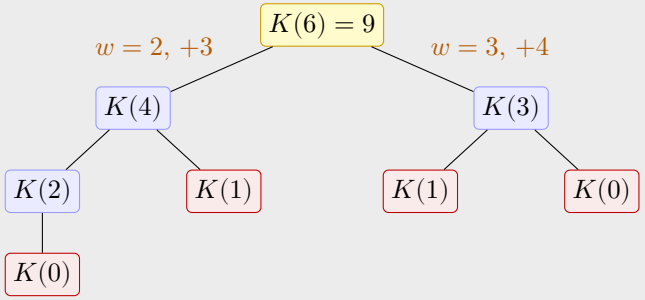
$$K(0) = 0 \qquad K(c) = \max\left(\{0\} \cup \{\, K(c - w_i) + v_i \mid 0 < w_i \leq c \,\}\right)$$




**items** $(w, v)$:
$(2, 3)$ $(3, 4)$

# Dynamic Programming: Knapsack

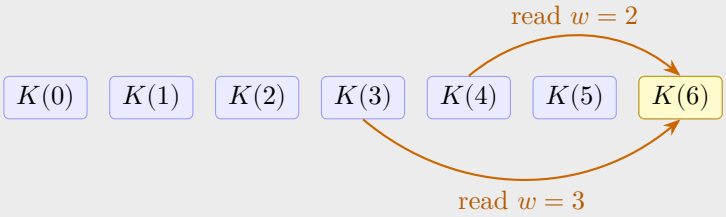
$$K(0) = 0 \qquad K(c) = \max\left(\{0\} \cup \{\, K(c - w_i) + v_i \mid 0 < w_i \leq c \,\}\right)$$




**items** $(w, v)$:
$(2, 3)$ $(3, 4)$

A hylomorphism, but it **recomputes**: exponential.

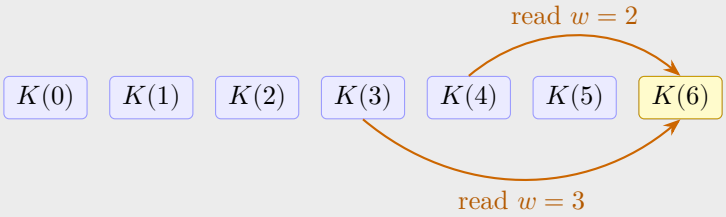
# Remember, Don't Recompute




- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

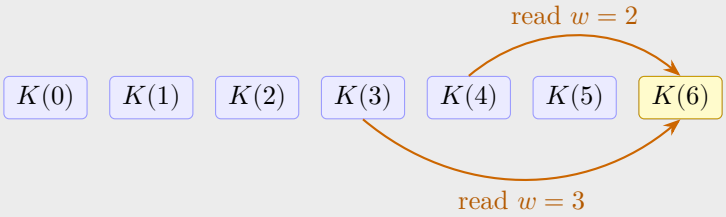
# Remember, Don't Recompute




- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.
- A law: recursion over *items* = recursion over *capacities* with a table, if each step *looks up* its subproblems.

R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

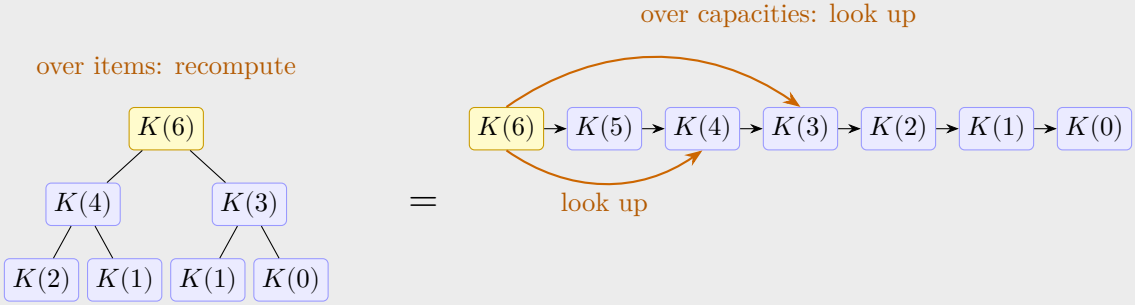
# Remember, Don't Recompute




- **Dynamorphism**: a hylomorphism that keeps a *table* of earlier results.
- A law: recursion over *items* = recursion over *capacities* with a table, if each step *looks up* its subproblems.
- Exponential $\Longrightarrow$ one entry per capacity.

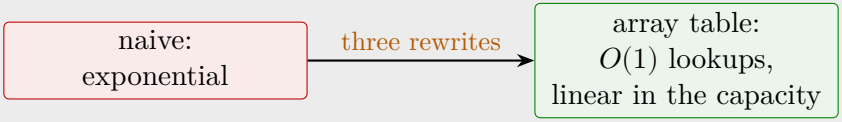
R. Hinze, N. Wu, J. Gibbons. **Conjugate Hylomorphisms, or: The Mother of All Structured Recursion Schemes**. POPL 2015.

# From Recomputation to a Table, by Rewriting



# From Recomputation to a Table, by Rewriting

```
Derive knapsack_arr in (knapsack_naive wvs =e morph_of knapsack_arr)
  as knapsack_arr_calc.
Proof.
  unfold knapsack_naive.
  rewrite (reenumerate (U := ATab int) (alg := knap_alg) knap_look).
  rewrite (look_tbl_with fast_drop_walk).
  rewrite dyna_memo.
  finished.
Qed.
```



# Knapsack, Step by Step

```
knapsack_naive wvs
```

# Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
```

# Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
=e  (* reenumerate: recurse on capacities, look results up *)
  dyna (knap_alg wvs \o look_tbl) out_int_c
```

# Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
=e  (* reenumerate: recurse on capacities, look results up *)
  dyna (knap_alg wvs \o look_tbl) out_int_c
=e  (* look_tbl_with: jump straight to the entry, O(1) *)
  dyna (knap_alg wvs \o look_with (w:=fast_drop) fast_drop_walk) out_int_c
```

# Knapsack, Step by Step

```
  knapsack_naive wvs
=e  (* unfold: the exponential specification *)
  hylo naive_alg (naive_c wvs)
=e  (* reenumerate: recurse on capacities, look results up *)
  dyna (knap_alg wvs \o look_tbl) out_int_c
=e  (* look_tbl_with: jump straight to the entry, O(1) *)
  dyna (knap_alg wvs \o look_with (w:=fast_drop) fast_drop_walk) out_int_c
=e  (* dyna_memo: build each table entry once *)
  (mhead \o l_out) \o
    hylo (memo_alg (knap_alg wvs \o look_with (w:=fast_drop) fast_drop_walk))
      out_int_c
```

# Knapsack: What Comes Out

```
let knapsack_arr wvs x =
  head
    (let rec f x0 =
       if Uint63.equal x0 (Uint63.of_int (0))
       then single
              (imax (Uint63.of_int (0))
                (let rec f0 = function
                 | [] -> min_int
                 | _ :: t -> imax min_int (f0 t)
                 in f0 wvs))
       else let r = f (Uint63.sub x0 (Uint63.of_int (1))) in
            push
              (imax (Uint63.of_int (0))
                (let rec f0 = function
                 | [] -> min_int
                 | h :: t ->
                   imax
                     (match fast_drop (fst h) r with
                      | Some a -> Uint63.add (head a) (snd h)
                      | None -> min_int)
                     (f0 t)
                 in f0 wvs))
              r
     in f x)
```

# Knapsack: What Comes Out

```
let rec jump n t =
  let { blk = b; top = i; prev = p } = t in
  if Uint63.le n i
  then Some { blk = b; top = (Uint63.sub i n); prev = p }
  else (match p with
        | Some q -> jump (Uint63.sub (Uint63.sub n i) (Uint63.of_int (1))) q
        | None -> None)

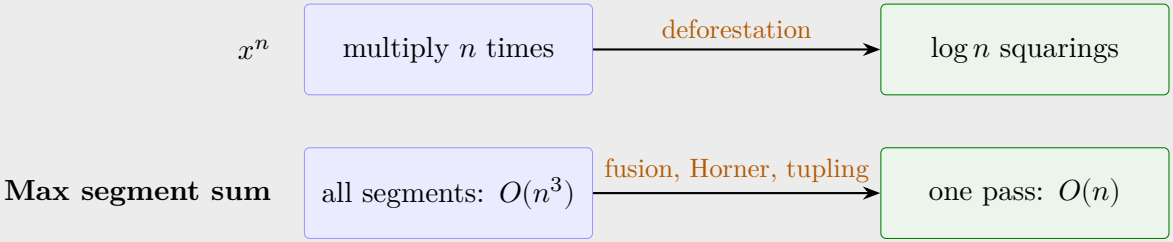
let fast_drop j t =
  if Uint63.equal j (Uint63.of_int (0))
  then None
  else jump (Uint63.sub j (Uint63.of_int (1))) t
```

`fast_drop`: jump straight into the right block of the array, $O(1)$. Knapsack is linear in the capacity.

# More Calculations

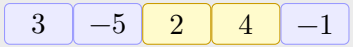


# More Calculations



# Maximum Segment Sum: Spec and Result

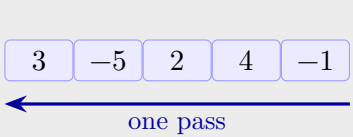
$$\mathsf{mss} = \mathsf{maximum} \circ \mathsf{map}\,\mathsf{sum} \circ \mathsf{segments}$$




best segment: $2 + 4 = 6$

# Maximum Segment Sum: Spec and Result

$$\mathsf{mss} = \mathsf{maximum} \circ \mathsf{map}\,\mathsf{sum} \circ \mathsf{segments}$$




best so far: 6

best starting here

Calculated: cubic $\Longrightarrow$ **one pass**, two numbers.

# Summary So Far

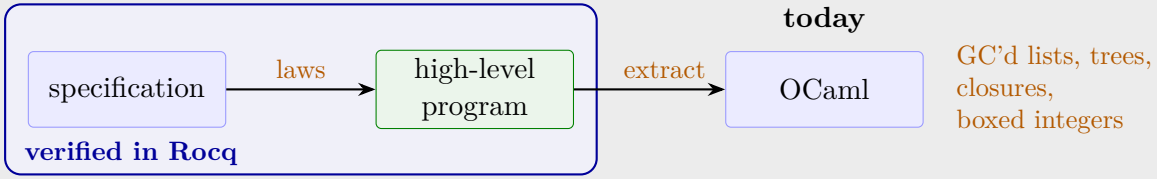
- A **library**: hylomorphisms and their laws in Rocq.
- Optimisations are **proofs by rewriting**.
- Termination proved **once per split**.
- **No extra axioms**; OCaml **without unsafe casts**.

D. Castro-Perez, M. Paviotti, M. Vollmer. **Program Optimisations via Hylomorphisms for Extraction of Executable Code**. ITP 2025. Library: `https://github.com/dcastrop/rocq-hylomorphisms`

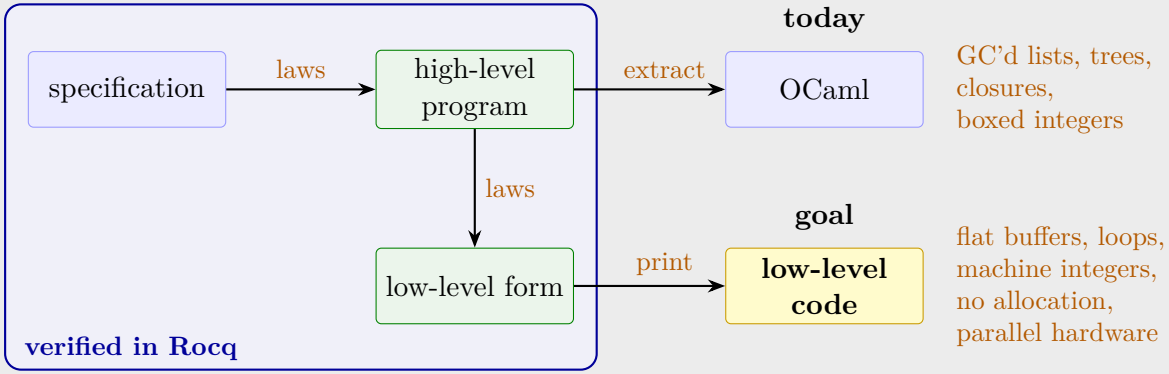
# Work in Progress

Down to Low-Level Code

# Two Ways Out of Rocq



# Two Ways Out of Rocq



## What We Are Working On

**Low-level**, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.

## What We Are Working On

**Low-level**, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

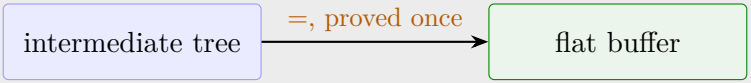
- You choose *laws*, not pointers and indices.
- Every step proved; the output is machine-level code.

## What We Are Working On

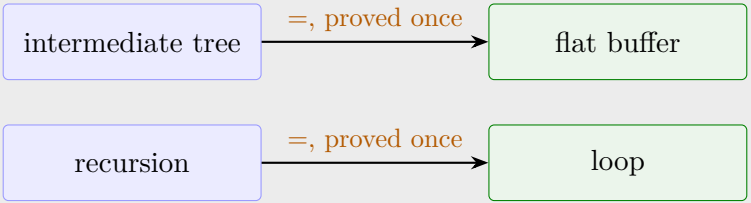
**Low-level**, **correct-by-construction** code,
extracted from **high-level calculation steps**,
possibly guided by a **cost model**.

- You choose *laws*, not pointers and indices.
- Every step proved; the output is machine-level code.
- A cost model says *which* equal program to pick.

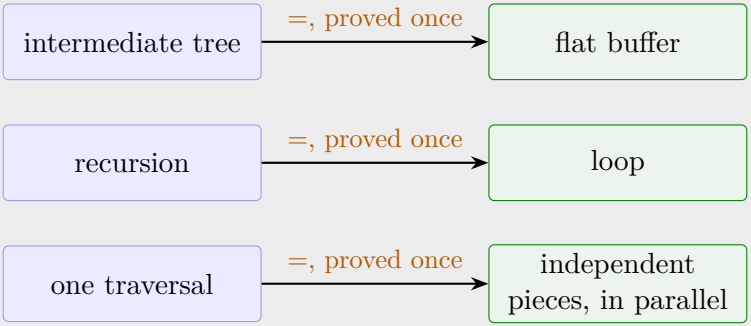
## Ongoing: Laws That Change Representation



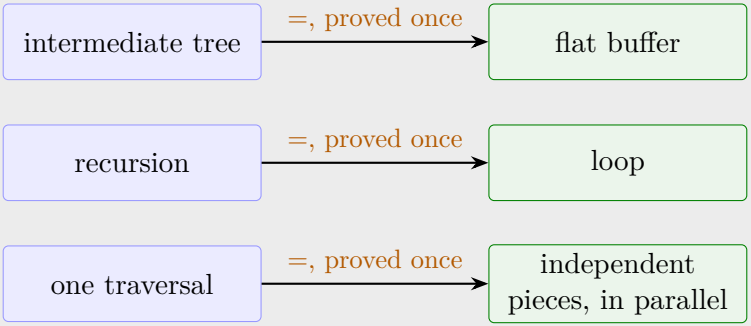
# Ongoing: Laws That Change Representation



# Ongoing: Laws That Change Representation

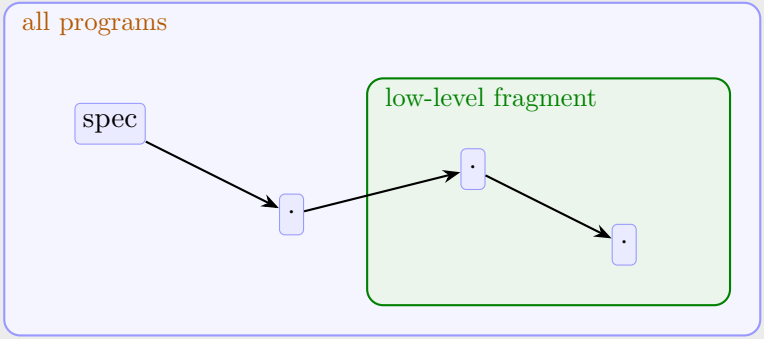


# Ongoing: Laws That Change Representation



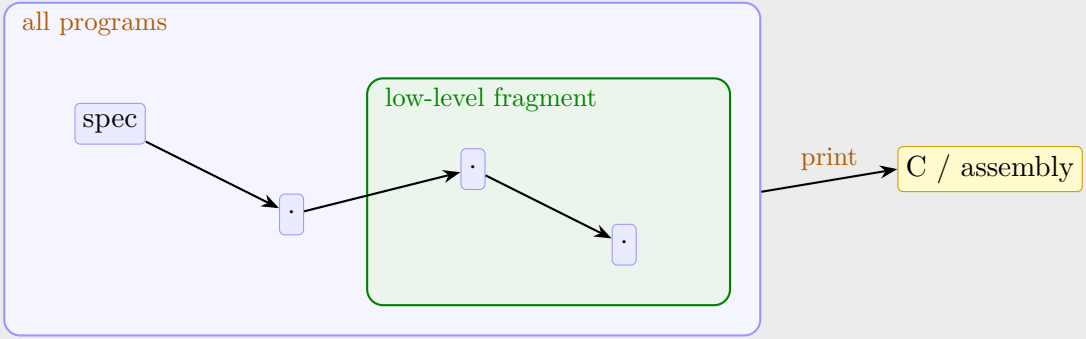

Today's laws change the *algorithm*; these change *where data lives* and *how work is scheduled*.

# Ongoing: A Target the Laws Never Leave



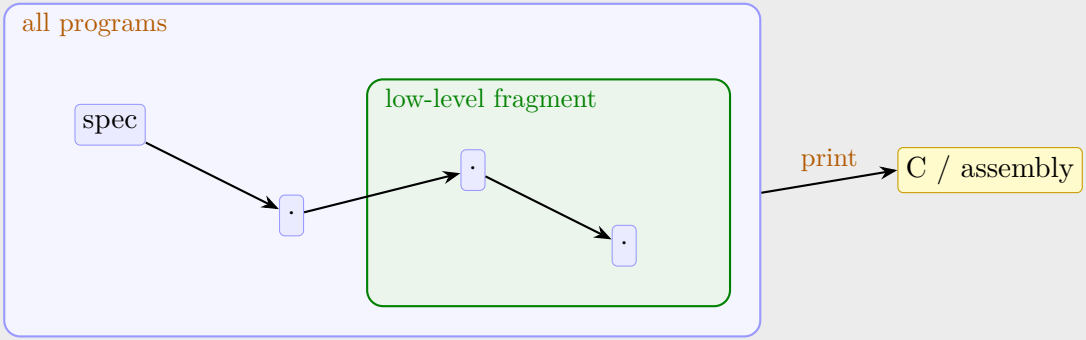

- Every law keeps programs **inside** the fragment.

# Ongoing: A Target the Laws Never Leave



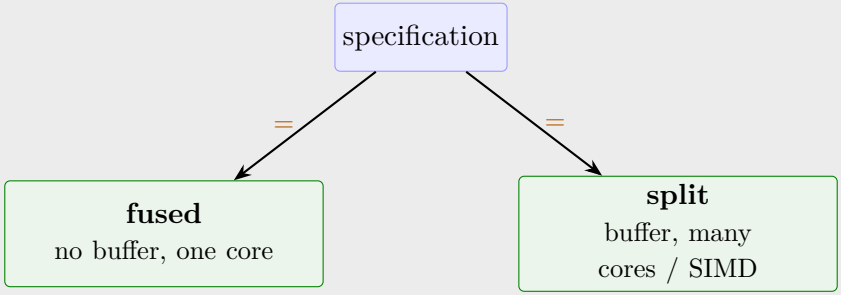

- Every law keeps programs **inside** the fragment.
- Only a **small printer** is trusted.

# Ongoing: A Target the Laws Never Leave

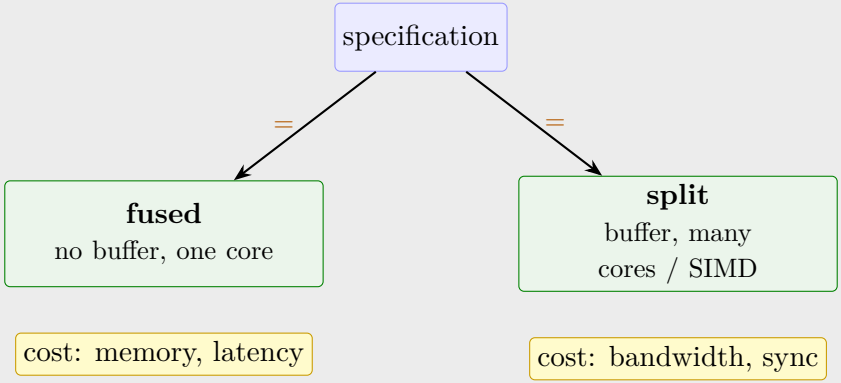



- Every law keeps programs **inside** the fragment.
- Only a **small printer** is trusted.
- Overflow and bounds: **side conditions** of the laws.

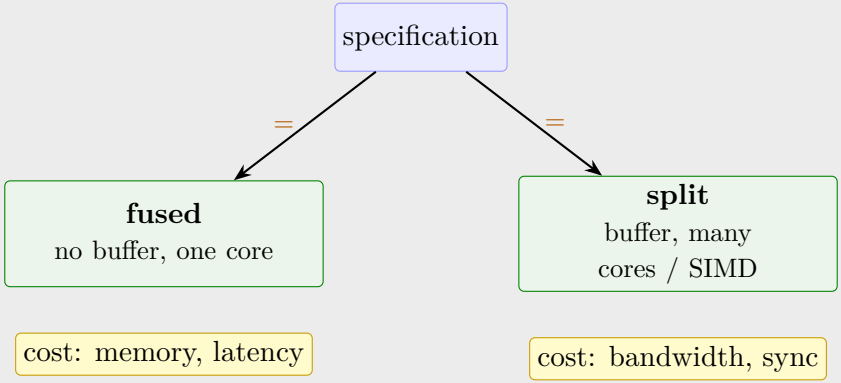
# Ongoing: A Cost Model



# Ongoing: A Cost Model

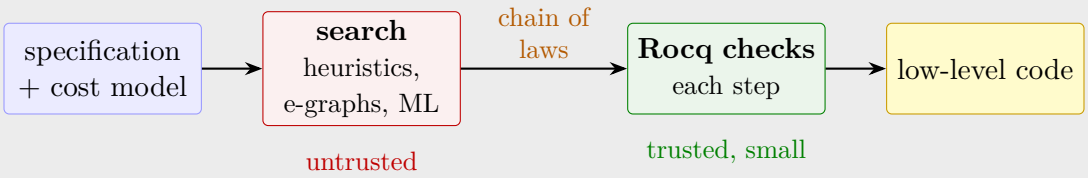


## Ongoing: A Cost Model



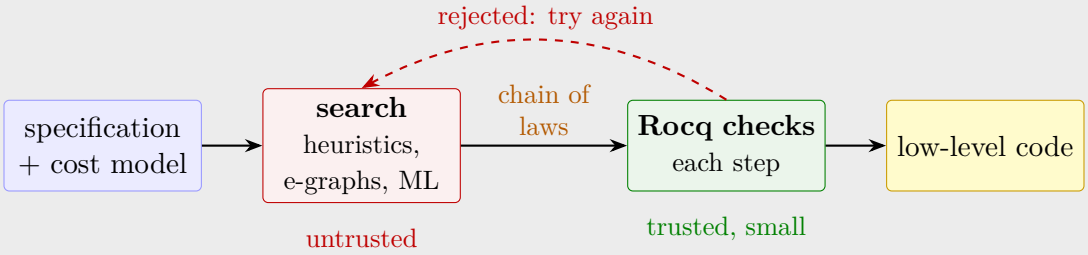

The laws say what is *equal*; a cost model says what is *better*.

# Untrusted Search, Trusted Checking




- The derivation is a **certificate**.

# Untrusted Search, Trusted Checking




- The derivation is a **certificate**.
- Search can be as clever, or as wrong, as it likes.

# Where We Are




**In Rocq, axiom-free:**

- Splitting index ranges $k$ ways
- Machine integers, no overflow (proved)
- Buffers, compared by their reads
- Tree reductions over ranges
- Writes to a buffer, driven by any split

**Not yet:**

- Laws that add or remove a buffer
- Each index written exactly once
- The low-level target and printer
- The cost model and search

## Summary

- Program optimisation as **algebra**: rewrite with laws.
- Divide and conquer as **hylomorphisms**: laws proved once.
- In Rocq: **machine-checked**, no extra axioms, clean OCaml.
- Ongoing: **low-level, correct-by-construction code** from high-level steps, guided by a **cost model**.

# Thank you!

Questions?

`https://github.com/dcastrop/rocq-hylomorphisms`

Based on joint work with Marco Paviotti and Michael Vollmer.